PART A
Experiment 9

A.1 :Aim

To study and implement system and application monitoring and alerting using Prometheus and Grafana(to create dashboards for visualizing important performance metrics such as CPU, memory, disk, and system availability.).

A.2 : Prerequisites

Students should have:

- Basic knowledge of DevOps concepts and software development life cycle (SDLC).
- Basic understanding of servers, applications, and system resources.
- Basic knowledge of Linux/Windows operating systems.
- Basic understanding of Docker and container concepts is desirable.
- Basic knowledge of networking concepts such as IP address, ports, and client-server communication.
- Familiarity with web browsers and basic command-line operations.

A.3: Lab Outcomes

After successful completion of this laboratory, students will be able to:

1. Understand the concepts of monitoring, metrics, dashboards, and alerting in a DevOps environment.
2. Install and configure Prometheus for collecting and storing system/application metrics. 3. Configure monitoring targets and exporters such as Node Exporter for collecting system-level metrics. 4. Use PromQL to query and analyze monitoring data collected by Prometheus.
5. Install and configure Grafana and connect Prometheus as a data source.
6. Create Grafana dashboards to visualize CPU utilization, memory usage, disk usage, network activity, and system availability.
7. Configure alerts based on predefined threshold conditions and analyze system performance.
8. Interpret monitoring dashboards and alerts to identify potential system or application issues.

A.4:Theory

PART B

(PART B: TO BE COMPLETED BY STUDENTS)

Roll No. A27	Name: Prajwal Zolage
Class: BE–AI&DS	Batch: A1
Date of Experiment:	Date of Submission:

B.1 Code Screenshot:

Docker-compose.yml

docker-compose.yml | prometheus.yml

C: > Users > suraj > Downloads > experiment9 > docker-compose.yml

```yaml
services:
  prometheus:
    image: prom/prometheus:latest
    container_name: prometheus
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - prometheus_data:/prometheus
    command:
      - "--config.file=/etc/prometheus/prometheus.yml"
    depends_on:
      - node-exporter
    restart: unless-stopped

  node-exporter:
    image: prom/node-exporter:latest
    container_name: node-exporter
    ports:
      - "9100:9100"
    restart: unless-stopped

  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    ports:
      - "3000:3000"
    volumes:
      - grafana_data:/var/lib/grafana
    depends_on:
      - prometheus
    restart: unless-stopped

volumes:
  prometheus_data:
  grafana_data:
```

Prometheus.yml

docker-compose.yml | prometheus.yml

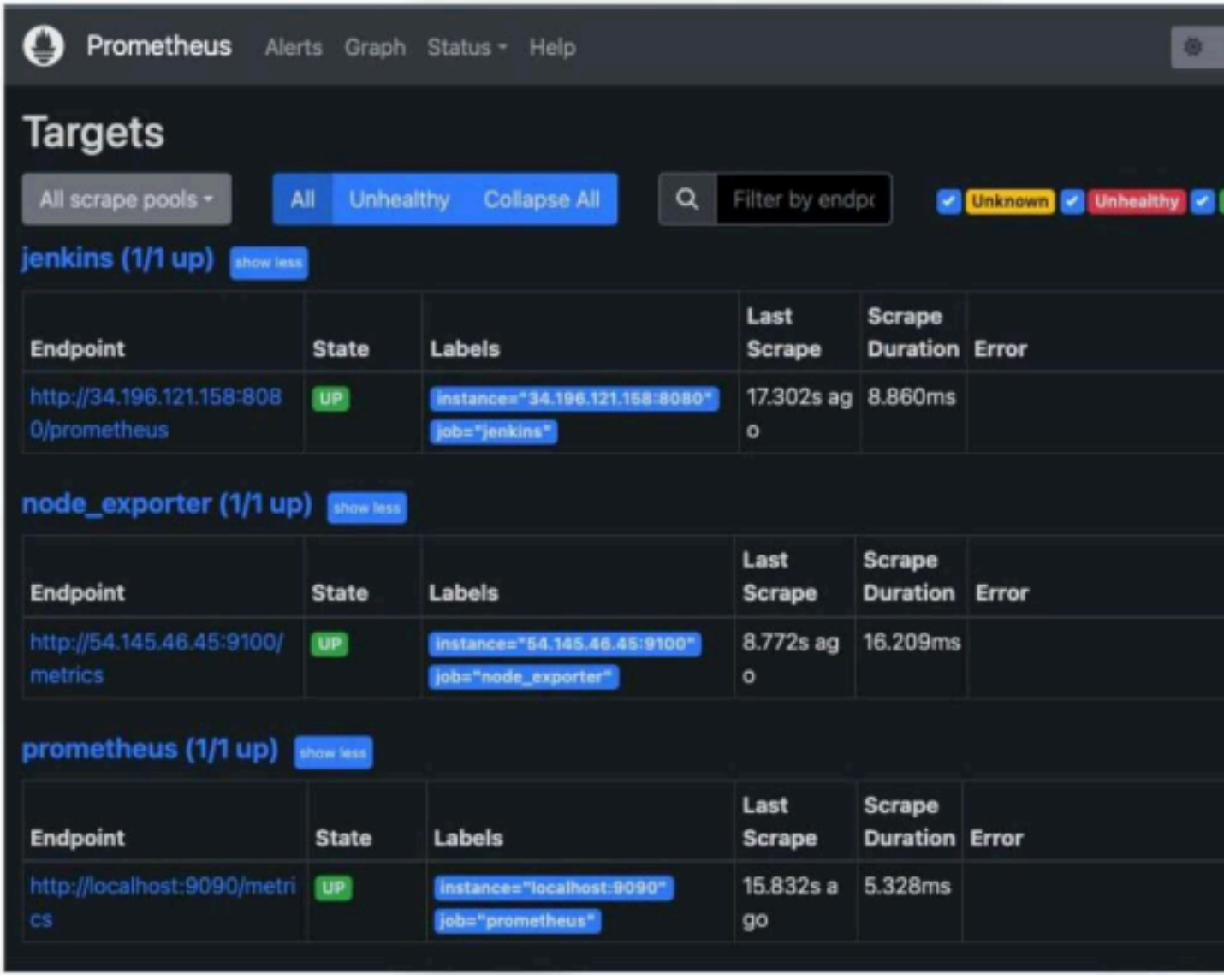
C: > Users > suraj > Downloads > experiment9 > prometheus.yml

```yaml
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: "prometheus"
    static_configs:
      - targets: ["prometheus:9090"]

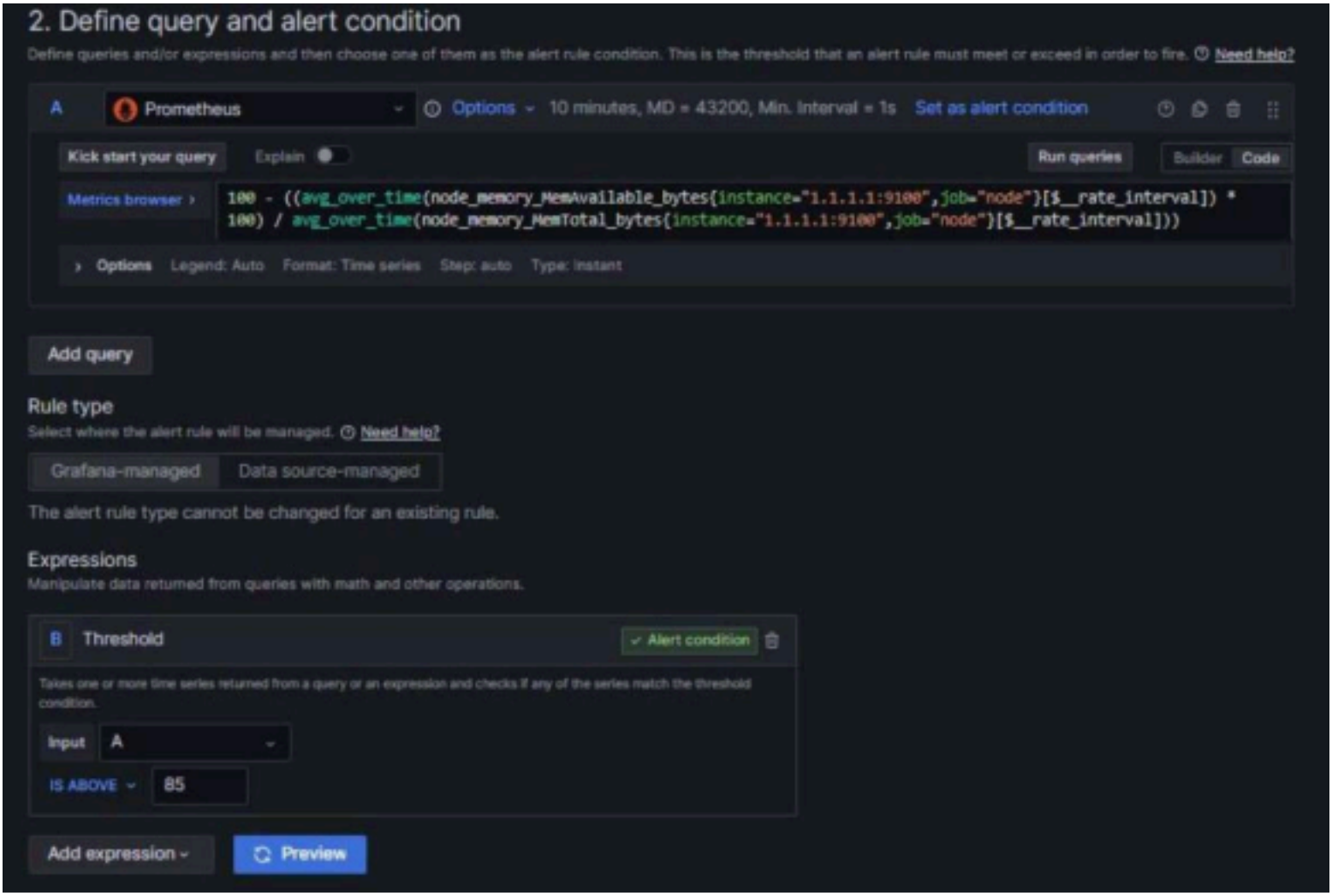
  - job_name: "node"
    static_configs:
      - targets: ["node-exporter:9100"]
```

B.2 Input and Output:



Grafanadashboard


Basic CPU / Mem / Disk Gauge
CPU Busy
18%
Used RAM Memory
80%
Used SWAP
54%
Used Root FS
76%
CPU System Load (1m a...
14%
CPU System Load (5m a...
4%
Basic CPU / Mem / Disk Info
CPU Cores
1
Total RAM
586.05 MiB
Total SWAP
500.00 MiB
Total RootFS
29.0 GiB
System Load (1m avg)
0.14
Uptime
31.9 weeks
Basic CPU / Mem Graph
CPU Basic
100%
75%
50%
25%
0%
21:00
22:00
23:00
00:00
01:00
Busy System
Busy User
Busy Iowait
Busy IRQs
Busy Other
Idle
Memory Basic
954 MiB
715 MiB
477 MiB
238 MiB
0 B
RAM Total
RAM Used
RAM Cache + Buffer
RAM Free
SWAP Used
Basic Net / Disk Info
Network Traffic Basic
20 KiB
10 KiB
0 B
-10 KiB
-20 KiB
-29 KiB
-39 KiB
Disk Space Used Basic



B.3 Conclusion:

The laboratory provides practical knowledge of monitoring and alerting using Prometheus and Grafana. Students learn to collect system metrics using Prometheus and exporters, analyze the collected data using PromQL, and visualize it through interactive Grafana dashboards. They also learn to configure alerts for detecting abnormal system conditions. This experiment demonstrates how monitoring and alerting support proactive issue detection, system reliability, performance analysis, and continuous monitoring in modern DevOps environments.